

Software Requirements and Design Document

For

Group 27

Version 1.0

Authors:

Talia L.

Makhtar N.

Nathaniel A.

Christopher K.

Carter M.

1. Overview (5 points)

EventPro is a web-based application designed to streamline event planning by integrating multiple event management tools into a single platform. The system allows users to manage guest lists, send invitations, track RSVPs, create and manage to-do lists (in a Kanban-style format), and track budgets. The platform features different access levels: regular users, event organizers, and administrators, each with varying permissions and functionalities.

The motivation behind EventPro is to simplify event planning, which can be overwhelming due to the need to coordinate multiple tasks and tools. Unlike existing platforms such as Eventbrite or RSVPify, EventPro combines invitation management, budgeting, and task collaboration into a single, user-friendly application tailored for both casual and professional users.

2. Functional Requirements (10 points)

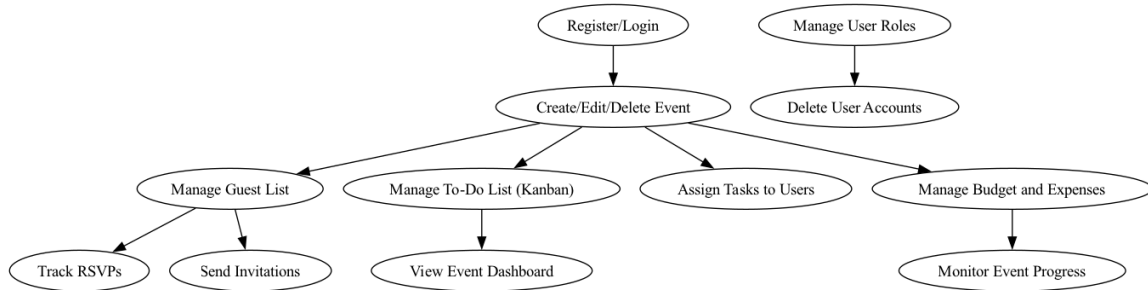
1. *Guest List Management & RSVP Tracking (High Priority)*
2. *Budget Tracker (High Priority)*
3. *To-Do List (High Priority)*
The website should allow event organizers to make a budget and allocate to different sections
4. *Event Creation & Management (High Priority)*
The application allows us to create an event with details like event name, date, location, and description.
5. *User Registration and Authentication (High Priority)*
The website has to allow users to register using an email and password.
6. *Calendar Integration (Low Priority)*
7. *User Interface Design (Low Priority)*

3. Non-functional Requirements (10 points)

The application should be reliable and users should always have access to any features allowed on their access level. Also user data and passwords should be encrypted and stored in a reliable persistent database. The app should also provide clear error messages to users when an action fails.

4. Use Case Diagram (10 points)

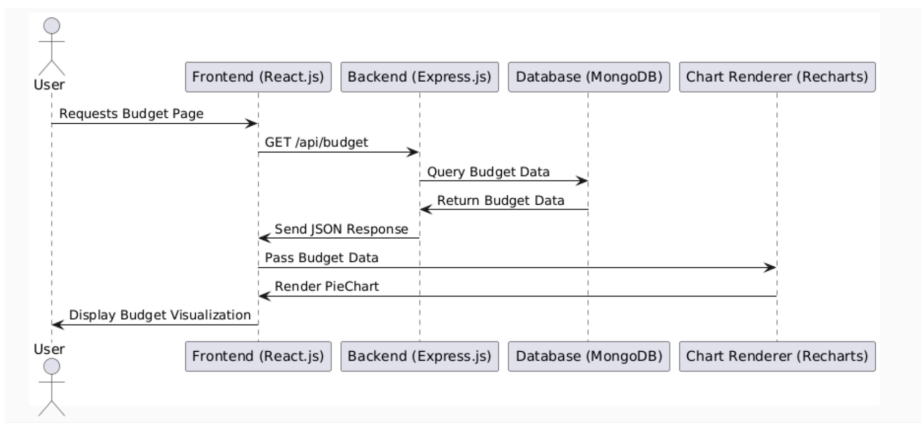
*This section presents the **use case diagram** and the **textual descriptions** of the use cases for the system under development. The use case diagram should contain all the use cases and relationships between them needed to describe the functionality to be developed. If you discover new use cases between two increments, update the diagram for your future increments.*

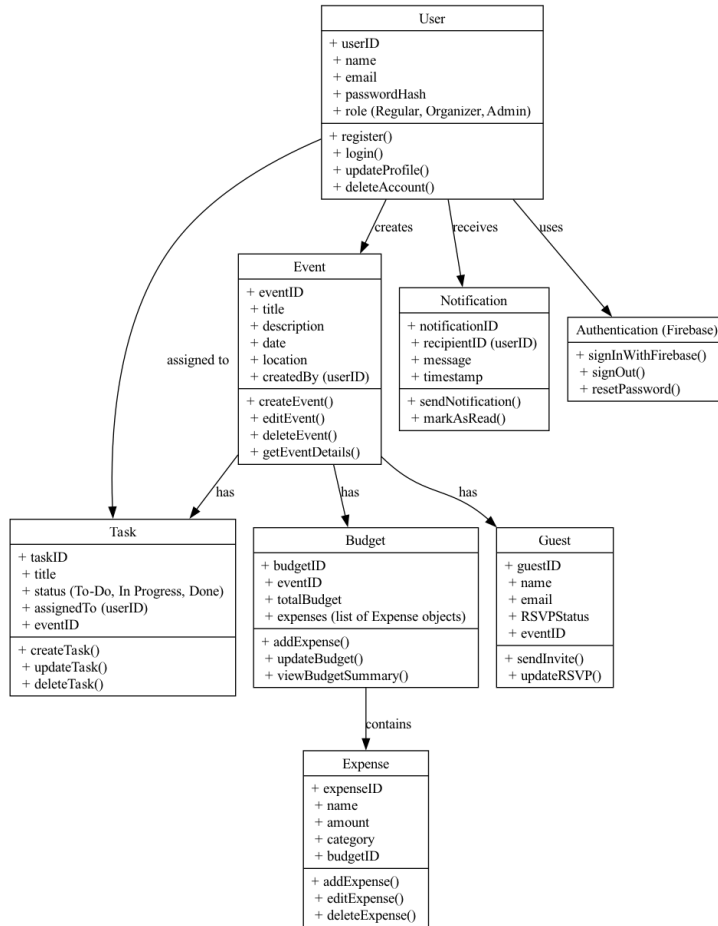


5. Class Diagram and/or Sequence Diagrams (15 points)

Class Diagrams show the **fundamental objects/classes** that must be modeled with the system to satisfy its requirements and **the relationships** between them. Each class rectangle on the diagram **must also include the attributes and the methods of the class** (they can be refined between increments). All the **relationships between classes and their multiplicity** must be shown on the class diagram.

A **Sequence Diagram** simply depicts **interaction between objects** (or **functions** - in our case - for non-OOP systems) in a sequential order, i.e. the order in which these interactions take place. Sequence diagrams describe how and in what order the objects in a system function.





6. Operating Environment (5 points)

The application will run on web browsers, more specifically on laptops and PCs. Our software components include Javascript, React.js, CSS, HTML, and some tailwind. We will also use API's like the google calender API and SendGrid

7. Assumptions and Dependencies (5 points)

The development of EventPro assumes that all third-party services, including MongoDB, Firebase, Google Calendar API, and SendGrid, will remain available and functional throughout the project. It is also assumed that React.js, Express.js, and Tailwind CSS will be sufficient for building a responsive and efficient web application. Additionally, the project relies on Jest and Postman for functional testing, Lighthouse for performance analysis, and ESLint for code quality enforcement. Any changes to these tools, such as API restrictions, pricing updates, or deprecations, could impact the development process and require adjustments to the system architecture or testing strategies